Author: Andrey Pautov. Published on Medium

What AI-Assisted Offensive Work Actually Means for Your Detection Program: A Practitioner's Dependency Audit

What the public record supports, what it does not, and how to audit the analytics most likely to fail under cheaper variation.

By Andrey Pautov — April 2026

You know this failure mode already. A rule tagged "behavioral" stops firing after a renamed binary, a parser change, or a missing command-line field. The label survived. The coverage did not.

[Documented] The public record through April 2026 shows actors using AI for research, malicious scripting, payload work, and a smaller set of cases where AI appears inside live tooling or operational flow. (3, 4, 5, 6, 7)

[Inferred] That pattern likely reduces labor for some forms of code variation and payload work.

[Inferred] That does not prove your analytics are failing. It does justify an audit of dependencies the adversary controls and can now vary more cheaply. The audit method here is a dependency-inventory and validation method. It does not replace Bianco or CTID.

Table of Contents

1. What the Public Record Actually Says (and What It Doesn't)
2. The Two Frameworks You Need and Why They Answer Different Questions
3. The Dependency Map: How to Classify What You Have
4. The Classification Rule: Apply It Per Dependency, Not Per Analytic
5. The Local Validation Test: Three Variants, Three Measurements, One Decision
6. Prioritization: Where to Start When You Have 300 Analytics

7. What This Does Not Tell You
8. Conclusion
9. References

---

1. What the Public Record Actually Says (and What It Doesn't)

OpenAI, February 14, 2024

- [Documented] OpenAI reported state-affiliated actors using LLMs for research, translation, scripting help, debugging, and phishing-related content generation. (3)
- [Inferred] The report did not establish novel intrusion techniques, measured defender-side degradation, or end-to-end autonomous compromise.
- [Inferred] The source reflects OpenAI-visible activity. It is not a census of actor behavior off that infrastructure.

Microsoft Security, February 14, 2024

- [Documented] Microsoft described the same pattern. AI was a productivity tool for threat actors. It supported research, translation, scripting, and phishing preparation. (4)
- [Documented] Microsoft said plainly that it had not observed especially novel or unique AI-enabled attack or abuse techniques. OpenAI said the same on the same date. (3, 4)
- [Inferred] This is still provider-visible activity and should not be treated as representative of actor behavior outside Microsoft-visible environments.

Google GTIG, January 29, 2025

- [Documented] GTIG reported use of Gemini for research, vulnerability work, malicious scripting, payload development, translation, and post-compromise support. (5)
- [Documented] GTIG also said it did not observe actors developing novel capabilities from that use.
- [Inferred] The source shows what Google could observe. It does not settle what actors do outside that visibility.

Google GTIG, November 5, 2025

- [Documented] GTIG's November 2025 tracker added public cases where AI appeared inside malware or operational workflows, not just in pre-operational support work. (6)
- [Inferred] The tracker did not establish ecosystem-wide prevalence, measured defender-side failure rates, or a collapse in detection durability.
- [Inferred] The source is provider casework. It is high signal. It is not representative sampling.

Microsoft Security, November 3, 2025

- [Documented] Microsoft reported SesameOp as a backdoor that used the OpenAI Assistants API for command and control. (7)
- [Inferred] The report did not establish that this channel was common, that most actors would copy it, or that existing analytics had already failed at scale because of it.
- [Inferred] The case matters because it creates a real dependency-mapping problem. It still reflects provider-visible activity on hosted infrastructure.

[Documented] Taken together, these sources show AI use in research, malicious scripting, payload work, and selected live operational cases.

[Inferred] They support a labor-reduction inference for some offensive support work, including some forms of code variation and payload preparation.

[Inferred] They do not show measured failure of your detection program. They do justify testing analytics that depend on adversary-mutable artifacts.

---

2. The Two Frameworks You Need and Why They Answer Different Questions

[Documented] Bianco's Pyramid of Pain asks one question: how much does it hurt the adversary when defenders force replacement? Hashes, IPs, and domains are low because replacement is cheap. Tools and TTPs are higher because replacement costs more time, retraining, and rework. (1)

[Documented] That is an attacker-cost model. It is not a measurement model for analytic quality. It does not tell you false-positive rate, parser resilience, maintenance burden, or whether your rule survives implementation change. (1)

[Documented] MITRE CTID's December 2024 work introduces spanning observables and decomposition methods. (2)

[Inferred] Those concepts provide defender-side vocabulary for reasoning about analytic durability under implementation variation. They still do not tell you which of your specific rules will fail first. That depends on the dependencies inside each implementation.

[Inferred] A defender can target a high-cost behavior with a low-durability rule if the implementation depends on narrow artifacts or weak telemetry.

[Inferred] These are separate questions. Conflating attacker replacement cost with defender analytic durability is a recurring analytical error in writing about AI and detection. A change can be expensive for the adversary and still break your rule. A change can be cheap for the adversary and still miss a spanning observable.

---

3. The Dependency Map: How to Classify What You Have

[Inferred] Teams often classify the analytic by name. They should classify the dependencies inside the implementation.

[Inferred] If cheaper variation matters anywhere, it matters first where coverage depends on implementation details the adversary can change. That is why the audit unit is the dependency, not the incident or the rule label.

[Inferred] Use four primary analytic dependency classes. Give each analytic dependency one primary class and record two other things alongside it: the controller of the dependency and any infrastructure prerequisites.

[Inferred] Apply the primary classes in order. If a dependency requires prior behavioral history for a specific entity, classify it as entity-baseline. If not, but the observable is structurally required across multiple implementations of the behavior, classify it as implementation-spanning. If not, but it still encodes a security-relevant action or state transition, classify it as behaviorally anchored. Otherwise classify it as artifact-dependent.

[Inferred] A primary class is a prioritization aid, not a claim that the rest of the dependency map is secondary. If two dependencies are equally necessary to rule behavior, record them separately instead of forcing one headline label.

[Inferred] Infrastructure prerequisites stay outside the four analytic classes. They still belong in the review because they confound local validation and often explain rule failure before adversary variation does.

Class 1 — Artifact-dependent

[Inferred] An artifact-dependent dependency requires an exact string, hash, path, tool name, mutex, user agent, or narrow regex fragment. In Windows process telemetry, that is `process_name = "mimikatz.exe"` in Sysmon Event ID 1 or Windows Security Event ID 4688. In proxy or firewall logs, that is an exact user agent or exact destination path.

[Inferred] If the adversary controls the artifact, the dependency is brittle under cheap variation. A rule on `process_name = "rundll32.exe"` plus a fixed path is not the same as a rule on a structurally required API call.

[Inferred] Some artifact dependencies are not adversary-mutable. They are third-party-constrained. A proxy rule on `dst_domain = api.openai.com` or a path tied to a vendor API is still artifact-dependent. The vendor controls that artifact, not the adversary. Treat it as stable only where the endpoint, request path, or client markers remain observable, and until the vendor changes them. Do not treat it like a renamed binary.

Class 2 — Behaviorally anchored

[Inferred] A behaviorally anchored dependency encodes a security-relevant action or state transition. Examples include a document-handling process spawning a script interpreter, a service account logging on interactively, or a non-system process reading LSASS memory.

[Inferred] In process telemetry, a rule on a document-handling process spawning a script interpreter is behaviorally anchored when it tries to encode document-to-script execution rather than one exact binary name. In network telemetry, a sequence of repeated outbound API interaction followed by local command execution is behaviorally anchored.

[Inferred] These dependencies may survive tool-name changes when the implementation does not gate on a specific parent, interpreter family, or exact field value. They still fail when the implementation is much narrower than the label suggests.

Class 3 — Implementation-spanning

[Documented] MITRE CTID's spanning-observable idea points to dependencies that survive multiple implementations of the same behavior because the observable is structurally required by the behavior itself. (2)

[Inferred] In practice, this means the dependency is anchored to what the adversary must do, not what they happened to name the binary. With API-call or EDR telemetry, a strong example across many user-mode

implementations is a process calling `OpenProcess` with `PROCESS_VM_READ` on `lsass.exe` and then calling `ReadProcessMemory`, regardless of process name or path.

[Inferred] These are often the most durable dependencies when the required telemetry is available and stable. They usually need EDR or API-call telemetry. They do not usually come from Windows Security Event ID 4688 alone.

Class 4 — Entity-baseline

[Inferred] An entity-baseline dependency requires prior behavioral history of a user, host, or service principal to score, suppress, or prioritize. In process telemetry, that may be "this host has never run PowerShell from Office." In proxy logs, that may be "this user has no prior Assistants API traffic."

[Inferred] These dependencies fail in four ways that are independent of adversary variation. Cold-start breaks new entities. Baseline contamination breaks already-compromised entities. Administrative change windows shift legitimate behavior. Environmental concept drift changes what "normal" means as new tools become common.

Recorded Separately — Infrastructure dependencies

[Inferred] Infrastructure dependencies are collection, parser, normalization, and correlation prerequisites. The adversary does not control them. A missing `CommandLine` field, a dropped Sysmon Event ID 1 stream, a changed Windows Security Event ID 4688 parser, or a broken join key can kill coverage before adversary variation matters.

[Inferred] In many real programs, infrastructure-controlled dependencies break as many or more analytics than attacker-side variation does. Record them separately. Fix them first when they are the actual failure point.

Primary class	What the dependency needs	Typical controller	Example	First review question
Artifact-dependent	Exact token, path, name, hash, or narrow regex	Adversary-mutable or third-party-constrained	`process_name = "mimikatz.exe"`; `dst_domain = api.openai.com`	Who controls the artifact?
Behaviorally anchored	Security-relevant action	Mixed	Office process spawning script interpreter	Does the implementation match the label?

Primary class	What the dependency needs	Typical controller	Example	First review question
	or state transition			
Implementation-spanning	Structurally required observable across variants	Usually harder for the adversary to change without changing behavior	LSASS read via API-call sequence	Is the observable required by the behavior?
Entity-baseline	Prior history for a user, host, or service principal	Environment-controlled	"No prior Assistants API traffic"	Is the baseline still trustworthy?

---

4. The Classification Rule: Apply It Per Dependency, Not Per Analytic

[Inferred] Most real analytics mix dependency classes. Do not classify the whole analytic. Classify each dependency.

[Inferred] Splunk Security Content's "Malicious PowerShell Process - Encoded Command" is a good example because the title sounds broader than the implementation really is. The example below is illustrative and only partially replayable from the published dataset. (8)

Entry	Primary class or prerequisite	Controller	Supporting constraint	First failure point
`process_name` or `original_file_name` identifies the PowerShell family	Artifact-dependent	Mixed: adversary-mutable plus metadata-dependent	PowerShell family identification must survive normalization and metadata handling	Loss of PowerShell family identification
Command-line regex matches `EncodedCommand` and shortened forms	Narrow behaviorally anchored	Adversary-mutable	Exact switch forms still matter	Variant outside the regex set

Entry	Primary class or prerequisite	Controller	Supporting constraint	First failure point
`CommandLine` field capture in the pipeline	Infrastructure prerequisite	Infrastructure-controlled	Parser and data-model preservation	Truncation, parser loss, or data-model change

[Inferred] Dependency 1 is artifact-dependent. A simple rename may not break it if `original_file_name` still resolves to the PowerShell family. The gate fails when PowerShell family identification no longer survives the variation, or when the actor shifts to a different launcher.

[Inferred] Dependency 2 is a narrow behaviorally anchored dependency because it tries to encode "PowerShell executing an encoded payload" as a security-relevant action, but only in one expression. It is not a broad detection of encoded execution across launchers or interpreters.

[Inferred] It still carries supporting constraints because the full command line must be present and the switch forms still matter. (8)

[Inferred] The `CommandLine` field is not an analytic class here. It is an infrastructure prerequisite. If that field is missing, truncated, or normalized differently, the analytic loses coverage before the adversary changes anything.

[Inferred] The key finding is simple. The label says "Malicious PowerShell Process." That sounds broader than the implementation. The implementation still leans on artifact-dependent PowerShell family identification. A team that treats this as durable coverage without testing is trusting the label, not the implementation.

[Inferred] The same dependency map helps on SesameOp. Microsoft described a backdoor using the OpenAI Assistants API for command and control. (7)

- [Inferred] One dependency is artifact-dependent and third-party-constrained: vendor API endpoints, request paths, and default SDK or user-agent markers when those are visible. The adversary does not control `api.openai.com`. OpenAI does. Treat these as stable only at telemetry points where the endpoint, request path, or client markers remain observable.
- [Inferred] One dependency is behaviorally anchored: an endpoint process shows repeated Assistants API sessions correlated with local command execution or file operations. That sequence matters more

than the client string, but it only exists where the SOC can correlate endpoint activity with network or proxy metadata.

- [Inferred] One dependency is entity-baseline: a host or identity with no prior Assistants API traffic suddenly shows this pattern. That signal has limited shelf life because enterprise LLM adoption creates concept drift.

[Inferred] The same incident therefore needs three review paths. Third-party-constrained artifacts need change tracking. Behaviorally anchored dependencies need sequence testing. Baseline dependencies need expiration and recalibration rules. Do not assign one shelf-life label to the whole detection package.

---

5. The Local Validation Test: Three Variants, Three Measurements, One Decision

[Inferred] This is a small-team test, not a research program.

[Inferred] The time estimate below assumes the team already has a replayable test harness or lab path into the same normalized pipeline the analytic uses. Building that path is separate work and often costs more than the first round of rule testing.

Inputs

- [Documented] One published analytic with stated telemetry prerequisites.
- [Documented] One baseline test case with the telemetry fields or event sequence the analytic requires. Use a real case when possible. If it is constructed from published attack data, state the provenance.
- [Inferred] Two behavior-preserving variants that change one implementation layer each.

Constraints

- [Inferred] Keep the objective, execution phase, and required privilege level constant across all three cases.
- [Inferred] Vary one layer per pass. Good first-pass layers are process family, command-line form, parent process, path, or client library.
- [Inferred] Hold telemetry source, parser version, normalized field mapping, and correlation window constant. If these drift, you are measuring collection failure, not implementation variation.

Measurement	Allowed values	What to record
Trigger retention	Yes / No	Did the analytic fire?
Analyst usefulness	High / Moderate / Low / None	How much triage work is needed?
Maintenance effort	None / Low / Moderate / High	What engineering lift restores coverage?

Analyst usefulness	Meaning
High	The alert is triageable without reconstruction.
Moderate	The alert is usable, but needs added context or manual reconstruction.
Low	The alert is weak and hard to separate from benign activity.
None	The analytic no longer produces a useful alert.

Maintenance effort	Meaning
None	No analytic change required
Low	Rule edit only, no new data source, under 4 hours
Moderate	Logic restructuring or normalization change, 4 hours to one sprint
High	New telemetry source, vendor dependency, or detection class replacement

[Inferred] `High` is not just a larger sprint. A new detection class or new telemetry source can mean weeks of work. Treat it as an architectural change, not a tuning task.

[Inferred] Using Splunk's published Encoded Command rule and Splunk attack data as the baseline case, the readout below is the right shape for a local test. (8, 9)

[Inferred] The second non-baseline case below is schematic. It represents the point where the PowerShell family no longer resolves in the identifying fields the rule uses.

Case	Layer changed	Trigger retention	Analyst usefulness	Maintenance effort	Failure point
Baseline	None	Yes	High	None	None

Case	Layer changed	Trigger retention	Analyst usefulness	Maintenance effort	Failure point
Variant 1	Command-line form	Yes	High	None	None
Variant 2	PowerShell family identity	No	None	Moderate	Loss of `process_name` / `original_file_name` family resolution

Decision rule

1. [Inferred] If trigger retention is `Yes` for both non-baseline variants, classify the tested dependency as surviving this variation distance. Do not call it spanning from two variants alone.
2. [Inferred] If trigger retention is `No` for either non-baseline variant, the tested dependency or prerequisite is confirmed not to survive that variation distance. Record whether the break was artifact-driven, infrastructure-driven, or due to a narrower implementation than the label implied.
3. [Inferred] If results are mixed, do not average them. Record the failed variant, the broken dependency, and the maintenance effort. The failure is the signal.
4. [Inferred] If only one non-baseline variant is complete, record the result as incomplete. Do not classify from one data point.

[Inferred] In a three-engineer team, one engineer can build variants, one can verify telemetry parity and parser consistency, and one can score usefulness and maintenance effort and update the inventory.

---

6. Prioritization: Where to Start When You Have 300 Analytics

1. Pull every analytic tagged or documented as `durable`, `high-confidence`, `TTP-based`, or `behavioral`. [Inferred] These are the highest-risk mislabeling candidates. Teams suppress or deprioritize alerts from rules they think are solid. A mislabeled tripwire in this tier can be suppressed, deprioritized, or misinterpreted in incident review. Record the rule ID, owner, tag source, and current suppression or severity settings. Decide which ones enter the first audit tier.
2. As a starting default, test process-execution analytics before network analytics in mixed EDR environments. [Inferred] Process telemetry normalization and process-family mapping diverge earlier across EDR vendors and versions than coarse network metadata. If your network controls already

expose stable third-party-constrained domains or request paths, those may be faster first-pass audits. Record the telemetry source for each rule. Record whether it depends on Sysmon Event ID 1, Windows Security Event ID 4688, vendor EDR fields, or a normalized data model. Decide the first batch from that map.

3. Build the dependency map before you build variants. Record each dependency, its class, and its controller: adversary-mutable, third-party-constrained, or infrastructure-controlled. Decide which single-layer variants are valid for that rule. If you skip this step, you are guessing.
4. Run the three-variant test. Record trigger retention, analyst usefulness, maintenance effort, and the exact failure point. Decide one of three outcomes for each dependency: survives this variation distance, narrower than label, or incomplete pending more testing.
5. Re-examine every entity-baseline dependency outside the three-variant test. Record cold-start risk, contamination risk, administrative change-window risk, and concept-drift risk. Decide whether the baseline is still useful, needs tighter scoping, or should stop driving priority.

[Inferred] For simple single-source endpoint analytics, one dependency map plus one three-variant test may take about two to four hours when the test harness already exists. Multi-source correlations and baseline-heavy analytics take longer. A program with 300 analytics therefore represents roughly 600 to 1,200 engineering hours at the low end once the harness, replay path, and normalized telemetry access already exist. Start with the highest-risk tier. Do not try to audit everything at once.

---

7. What This Does Not Tell You

[Inferred] This method tests simple, single-layer implementation variation first. Real adversaries combine layers. They rename the binary, change the path, swap the parent, and lose the command line in one move. Multi-layer testing should follow single-layer testing. It should not replace it.

[Inferred] AI does not create artifact mutability. Adversaries already had that option. The narrower point is that cheaper variation may make recovery, retesting, or evasion attempts cheaper after defensive friction. In many real programs, parser loss, field truncation, and normalization drift still break more analytics than adversary variation does.

[Inferred] The cited public record supports a labor-reduction inference for some forms of code variation, scripting, and payload work.

[Inferred] It does not show how often actors will spend that cheaper variation on detection evasion instead of target volume, fraud scale, or other offensive goals. The review is justified by mechanism plausibility. It is not justified by a measured effect size.

[Documented] These sources come from providers reporting on activity visible to their own platforms and investigations.

[Inferred] That creates provider selection bias.

[Speculative] Actors operating fully off-provider infrastructure may show different patterns. Absence of public evidence here is not evidence of absence there.

---

8. Conclusion

Do this this week: pull the first batch of analytics that your team already treats as durable coverage, and build a dependency map for them before you touch the test harness.

Stop assuming this: a rule labeled `behavioral`, `TTP-based`, or `high-confidence` is durable because the label says so. The implementation decides that. The dependency map exposes it.

This paper cannot tell you this: whether the adversaries in your environment will spend cheaper variation on evasion often enough to change your incident rate. Only local testing can answer that.

References

1. David Bianco, *The Pyramid of Pain* (2013)
2. Center for Threat-Informed Defense, *Summiting the Pyramid: Bring the Pain with Robust and Accurate Detection* (December 16, 2024)
3. OpenAI, *Disrupting malicious uses of AI by state-affiliated threat actors* (February 14, 2024)
4. Microsoft Security, *Staying ahead of threat actors in the age of AI* (February 14, 2024)
5. Google Threat Intelligence Group, *Adversarial Misuse of Generative AI* (January 29, 2025)
6. Google Threat Intelligence Group, *GTIG AI Threat Tracker: Advances in Threat Actor Usage of AI Tools* (November 5, 2025)

7. Microsoft Security, *SesameOp: Novel backdoor uses OpenAI Assistants API for command and control* (November 3, 2025)
8. Splunk Security Content, *Detection: Malicious PowerShell Process - Encoded Command* (updated March 25, 2026)
9. Splunk Security Content, *Encoded Powershell* attack data, including `explorer_spawns_windows-Sysmon.log` (published January 19, 2021)

↑ Back to top